Quo Vadis Backend?

Tymoteusz Huszcza

fajne tło, nie przyzwyczajać się →

Tył koniec?

Kto to jest Backend?

Co robi Backend?

Jak robi Backend?



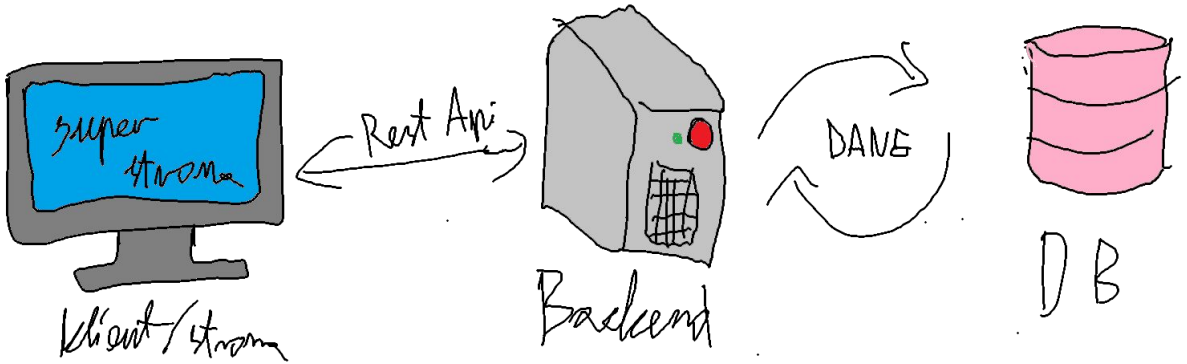
Kto...?

Warstwa aplikacji, odpowiadająca za komunikację DB - Strona.

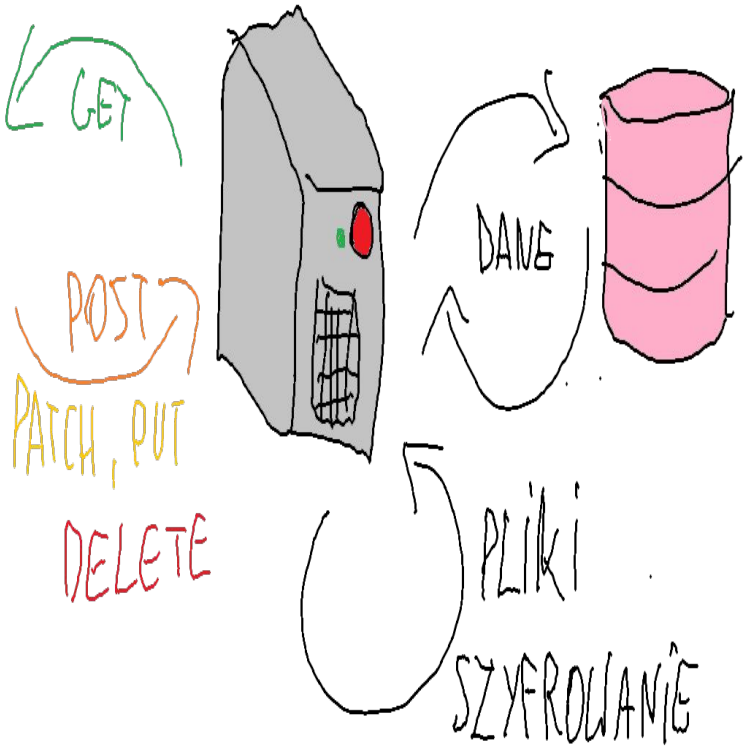
“Zaplecze” strony internetowej, pracuje wyłącznie na danych.

NIE JEST STRONĄ

Nie jest widoczny dla klienta



Co...?



Ma za zadanie pracować na danych, odbieranie, edycja, zwracanie, itd.

Obsługuje pliki, często też mailingi.

Często robi heavy lifting strony (a przynajmniej powinien), np. sortowanie danych, przekształcanie plików.

Jak...?

HTTP (API), CRUD, CDN, MICROSERVICES, SQL, SERVERLESS COMPUTING, BaaS FRAMEWORKI BACKENDOWE

CMS, M&M's, CDN, BaaS, SC, API...?

CMS - szajs typu wordpress

M&M's - taki snack

Cors - szajs blokujący nieupoważnione domeny (strony)

CDN (Content Delivery Network) - A content delivery network (CDN) or content distribution network is a geographically distributed network of proxy servers and corresponding data centers (Wikipedia) - szajs do optymalizacji zwracania danych

API - interfejs do komunikowania się między stroną a backiem, zazwyczaj używa się CRUD API (create, read, update, delete)

Microservices - zamiast jednego wielkiego, wiele małych backendów

SQL - język baz danych

Baas (Backend as Service) - funkcje backendowe udostępnione przez bigtechy

Serverless Computing - funkcje budowane na technologiach cloudowych, zamiast tworzenia pełnych backendów

Frameworki - technologie pomocnicze dla języków do tworzenia rzeczy

Po co…?

Rozdzielenie **backendu** i **frontendu** ma na celu poprawę efektywności, skalowalności i utrzymywalności aplikacji. Dzięki tej architekturze:

- **Niezależny rozwój**: Zespół frontendowy może pracować nad interfejsem, a backendowy nad logiką i danymi bez wzajemnych zależności. Można np. testować frontend z przykładowymi danymi JSON bez oczekiwania na gotowy backend.
- **Skalowanie i elastyczność**: Jeden backend może obsługiwać wiele klientów – strony internetowe, aplikacje mobilne, urządzenia IoT – bez konieczności powielania logiki biznesowej.
- **Łatwiejsze zarządzanie i utrzymanie**: Każdy zespół może skupić się na swojej specjalizacji – frontend z technologiami typu HTML, CSS, JavaScript, backend z językami takimi jak Java, Python czy PHP.
- **Wysoka wydajność i bezpieczeństwo**: Logika biznesowa i dostęp do baz danych pozostają w backendzie, co chroni dane przed bezpośrednim dostępem użytkownika. Frontend tylko prezentuje dane i przesyła żądania.
- **Ciągłe wdrażanie (CI/CD)**: Niezależność umożliwia niezależne wdrażanie zmian w frontendzie i backendzie, co przyspiesza rozwój i zmniejsza ryzyko błędów.

To podejście jest standardem w nowoczesnej开发 aplikacji webowych i mobilnych.

Odpowiedź wygenerowana przez AI. Prosimy zweryfikuj najważniejsze fakty.

REST API

REST API wykorzystuje protokół HTTP do komunikacji klient - serwer, przez różne metody i URL'e (endpointy). Nie przechowuje stanu klienta. Komunikuje się głównie JSON'ami.

Metody:

GET - dej mnie dane;

POST - trzymaj mordko dane;

PUT - we mi zmień tu wszystko;

PATCH - we mi zmień to tu o;

DELETE - usuń to, teraz;

WebSockets API

Wysyłanie i nasłuchiwanie eventów, by operować na danych w czasie rzeczywistym (np. chaty)



HTTP Responses

API powinno zwracać odpowiedni kod HTTP, przy okazji błędów i sukcesów. Mamy generalny podział:

- 1xx - wstrzymaj konie
- 2xx - jest git mordo (może nawet Hub??/?)
- 3xx - fajnie fajnie, teraz spadaj
- 4xx - panie, co pan robisz
- 5xx - sowwy, coś zepsuliśmy 👉👈



Podstawowe zabezpieczenia backendu

JWT - (zazwyczaj okresowy) kodzik do weryfikacji użytkownika, bardzo fajna rzecz

Hashowanie Haseł - kodowanie haseł za pomocą hashów

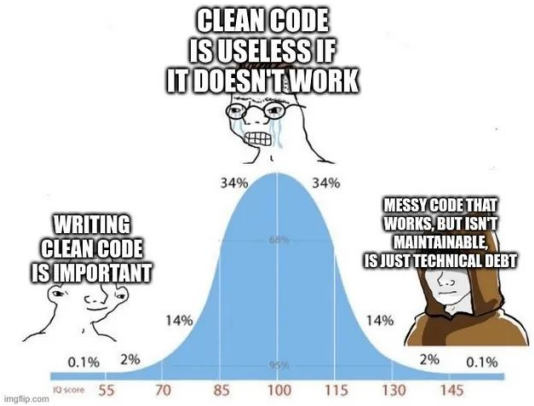
HTTPS - szyfrowanie zapytań

Role - każdy ziutek ma dostać tylko minimum które potrzebuje i nic więcej

Zapory sieciowe - usługi u firm hostingowych które bronią przed najpopularniejszymi atakami

Reverse Proxy - pośrednik umożliwiający przekierowania, izolowanie itd. ścieżek

Dobre praktyki



- Modularność - pamiętaj programisto młody, oddzielaj kontroler od routów (np. MCV)
- Logi - pamiętaj programisto młody, twórz logi by wiedzieć co się odwala
- Walidacja - pamiętaj informatyku młody, bez walidacji, [Object object] lub HTTP 500
- Clean Code - pamiętaj informatyku młody, SOLID DRY KISS

Pytania? nie? super.

Autor: Tymoteusz Huszcza

Źródła:

- Przeglądarka Brave,
- Google for devs,
- GeeksForGeeks
- Wikipedia